

学 号:	1023005510 1023004577 1023004609
------	--

武汉理工大学

软件工程实践（二）

学 院	计算机与人工智能学院
专 业	软件工程
班 级	软件 2302 班
姓 名	范奕轩
编 号	
指导教师	唐祖锴

2026 年 6 月 15 日

目录

一、任务概述	6
1.1 主要内容	6
1.1.1 核心玩法与交互系统构建	6
1.1.2 前后端分离架构与 Web 化改造	7
1.1.3 持久化存储与存档机制	7
1.2 任务目的	8
1.2.1 掌握面向对象设计原则与设计模式应用	8
1.2.2 体验全栈开发与前后端协同生态	8
1.2.3 深入理解软件工程实践与团队协作	9
二、小组任务计划	10
2.1 任务分析	10
2.1.1 系统架构演进与前后端分离通信设计	10
2.1.2 核心引擎的面向对象重构与业务拓展	11
2.1.3 2D 可视化前端与图形渲染机制	11
2.1.4 数据持久化与 ORM 框架集成	12
2.1.5 动态交互系统与沉浸式体验优化	12
2.2 项目架构与功能设计	13
2.2.1 总体架构设计	13
2.2.2 后端核心架构与设计模式应用	14
2.2.3 前端视图与交互引擎架构	14
2.2.4 核心功能模块设计	15
2.3 团队组成与分工	16
2.3.1 小组协作模式与管理策略	17
2.3.2 团队成员与具体分工说明	17
2.4 软件工具与项目约束	19
2.4.1 开发与工程化工具使用方案	19
2.4.2 项目管理策略与强制约束	21

三、小组软件过程 23

3.1 代码仓库	23
3.1.1 平台选择与仓库初始化	23
3.1.2 仓库目录规范与架构映射	24
3.1.3 权限管控与安全策略	24
3.2 软件版本计划	25
3.2.1 五大里程碑拆解与实施目标	25
3.2.2 基于 Milestone 的版本控制机制	27
3.3 开发分支模型与提交流程	27
3.3.1 基于成员实名的专属分支模型	27
3.3.2 语义化与原子化的代码提交流程	28
3.3.3 标准代码合并闭环 (Merge Workflow)	29
3.4 代码规范与检查流程	29
3.4.1 统一的代码编写规范	30
3.4.2 人工交叉代码审查机制	31
3.4.3 CI/CD 自动化代码门禁与测试检查	31
3.5 项目测试计划与实施流程	32
3.5.1 测试策略与工具选型	32
3.5.2 核心模块单元测试实施 (Unit Testing)	33
3.5.3 集成测试与系统端到端测试 (Integration & E2E Testing)	34
3.5.4 测试评估与质量保障机制	35
3.6 项目集成与发布规范	35
3.6.1 持续集成流水线	35
3.6.2 前后端全栈构建与打包策略	36
3.6.3 零配置部署与版本交付规范	37

四、小组成员任务报告 38

4.1 组长：范奕轩 任务分工及实施过程描述	38
4.1.1 承担的主要任务分工	38

4.1.2 实施过程描述	39
4.1.3 实施成果展示	41
4.2 成员二：王启赢 任务分工及实施过程描述	41
4.2.1 承担的主要任务分工	41
4.2.2 实施过程描述	42
4.2.3 实施成果展示	44
4.2.4 AI 辅助开发工具使用说明	44
4.3 成员三：何宇航 任务分工及实施过程描述	45
4.3.1 承担的主要任务分工	45
4.3.2 实施过程描述	45
4.3.3 实施成果展示	47
4.3.4 AI 辅助开发工具使用说明	48
五、实施过程问题记录与分析	49
5.1 前端图形渲染与物理引擎异常分析	49
5.1.1 透明 PNG 图像边缘渲染发光问题	49
5.1.2 物理碰撞引擎与场景边缘探测脱节	50
5.2 复杂对象网络与数据持久化危机	51
5.2.1 Hibernate 与 Gson 序列化引发的栈溢出	51
5.2.2 历史存档反序列化后的坐标漂移 Bug	51
5.3 敏捷协同与版本控制冲突事故	52
5.3.1 多分支错误回退与环境文件污染	52
六、任务总结	54
6.1 项目总体完成情况	54
6.2 技术能力与工程实践收获	55
6.3 团队协同与项目管理经验	55
6.4 不足与未来展望	56
【结语】	57

【参考文献】 58

- [1] 李晓蕙.网页游戏中界面设计的研究和应用[D].上海交通大学,2008. 58
- [2] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley. 58
- [3] Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall. 58
- [4] 黄敏珍.CMMI、敏捷开发和 DevOps 在项目管理实践中的应用[J].项目管理技术,2020,18(9):91-95. 58
- [5] Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd Edition). Addison-Wesley. 58
- [6] Vue3 详细教程:
https://blog.csdn.net/m0_55734030/article/details/127719835?fromshare=blogdetail&sharetype=blogdetail&sharerId=127719835&sharerefer=PC&sharesource=2403_88186781&sharefrom=from_link 58
- [7] Oracle. (2023). Java HTTP Server API Documentation. Retrieved from <https://docs.oracle.com/en/java/javase/17/docs/api/jdk.httpserver/com/sun/net/httpserver/HttpServer.html> 58
- [8] 戴建国,赵庆展,郭理,等.持续集成在项目开发中的应用研究[J].计算机工程与设计,2009,30(10):2573-2576.DOI:10.16208/j.issn1000-7024.2009.10.045. 58
- [9] You, E. (2023). Vue.js Documentation - Introduction. Retrieved from <https://vuejs.org/guide/introduction.html> 58
- [10] Beck, K., & Gamma, E. (2023). JUnit 4 Documentation. Retrieved from <https://junit.org/junit4/> 58
- [11] Jenkins CI/CD 流水线实战:
https://blog.csdn.net/weixin_45130293/article/details/144965697?fromshare=blogdetail&sharetype=blogdetail&sharerId=144965697&sharerefer=PC&sharesource=2403_88186781&sharefrom=from_link 58
- [12] Mozilla Developer Network. (2023). Fetch API - Web APIs. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API 58
- [13] 戴启铭,毛润丰,黄璜,等.DevSecOps:DevOps 下实现持续安全的实践探索[J].软件学报,2021,32(10):3014-3035.DOI:10.13328/j.cnki.jos.006276. 58
- [14] Conventional Commits. (2023). Conventional Commits 1.0.0. Retrieved from <https://www.conventionalcommits.org/> 58
- [15] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley. 58
- [16] Sommerville, I. (2015). Software Engineering (10th ed.). Pearson. 59

- [17] 邹益胜,丁国富,许明恒,等.实时碰撞检测算法综述[J].计算机应用研究,2008,(1):8-12. 59
- [18] Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. 59
- [19] SQLite 入门教程（非常详细）零基础入门到精通，收藏这一篇就够了：
https://blog.csdn.net/2401_84618514/article/details/145874453?fromshare=blogdetail&sharetype=blogdetail&sharerId=145874453&sharerefer=PC&sharesource=2403_88186781&sharefrom=from_link 59
- [20] 张美香,郝轶鸣.关键帧动画技术综述[J].山西广播电视大学学报,2009,14(5):55-56. 59
- [21] Beck, K. (2003). Test-Driven Development: By Example. Addison-Wesley Professional. 59
- [22] 张杨,黄厚宽.测试驱动开发及开发实践[J].计算机技术与发展,2006,(5):74-76+79. 59
- [23] Nystrom, R. (2014). Game Programming Patterns. Genever Benning. 59
- [24] 李涛.图形用户界面 GUI 的自动测试工具的研究[D].四川大学,2005. 59
- [25] Bauer, C., King, G., & Gregory, G. (2015). Java Persistence with Hibernate (2nd ed.). Manning Publications. 59
- [26] Hibernate 使用详解：
https://techfanyi.blog.csdn.net/article/details/139897654?fromshare=blogdetail&sharetype=blogdetail&sharerId=139897654&sharerefer=PC&sharesource=2403_88186781&sharefrom=from_link 59

一、任务概述

本实践项目基于经典文字冒险游戏“World of Zuul”（祖尔世界），结合现代 Web 技术、面向对象设计模式以及全栈开发架构，对其进行了深度的二次开发与系统重构。项目在保留原有地图探索核心逻辑的基础上，拓展了丰富的 RPG（角色扮演）元素，实现了前后端分离的 2D 可视化 Web 应用，并引入了关系型数据库进行状态持久化管理。

1.1 主要内容

本项目的主要内容涵盖了从底层核心逻辑重构到前端沉浸式 UI 设计的全链路开发，具体包含以下几个核心模块的构建与完善：

1.1.1 核心玩法与交互系统构建

在原有简单的文字导航基础上，系统大幅扩充了游戏机制，使之成为一个具备完整业务逻辑的 RPG 游戏系统。

（1）房间与地图导航机制

设计了多维度的房间结构（如剧院大厅、实验室、酒吧、神秘传送室等），并实现了动态的房间出口判定与边缘气泡预警系统。引入了随机传送机制（TransporterRoom），增加了地图探索的不可预见性。

（2）物理引擎与角色状态管理

构建了 2D 网格化的物理移动模型，支持玩家通过键盘（WASD/方向键）进行平滑移动与碰撞检测。引入了完整的角色状态（HP 血量、MP 魔法值、负重上限）以及多属性的物品系统（如具有恢复效果的魔法饼干、增加攻击力的道具剑等），并实现了拾取、丢弃、使用等精细化操作。

（3）动态战斗与 NPC 交互系统

独立开发了敌对 NPC（Saber）的 AI 寻路与警戒范围触发逻辑。实现了包括近战物理攻击、远程魔法子弹消耗、战败惩罚及胜利结算重置（满血

满蓝奖励) 在内的完整战斗状态机。

1.1.2 前后端分离架构与 Web 化改造

打破了传统 Java 控制台程序的局限，将单机游戏成功改造为基于 B/S (浏览器/服务器) 架构的 Web 级应用。

(1) 轻量级后端 API 服务

利用 Java 原生 `HttpServer` 搭建了非阻塞式的后端服务器 (`GameServer`)，通过定义清晰的请求处理器 (如 `CommandHandler`、`StateHandler`)，为前端提供 JSON 格式的 RESTful 风格接口，实现游戏状态与指令的异步通信。

(2) 2D 可视化与沉浸式 UI 渲染

前端基于现代响应式框架 (Vue 3+Bootstrap 5) 进行构建。彻底摒弃了原生纯文本输出，设计了高度沉浸式的“无边框做旧羊皮纸”风格对话框，并引入了双轨自定义古风字体 (`ActionFont` 与 `BodyFont`)。利用 CSS3 雪碧图动画解析 (`SpriteAnalyzer`) 实现了人物行走与攻击特效的流畅渲染。

(3) 全局中文语境深度本地化

在前端通过正则表达式对后端的底层英文指令及回显 (如超重警告、方向反馈、拾取动作等) 进行了全面的拦截与封装，配合彩色样式标签，实现了 100% 纯正中文的古风沉浸式交互提示。

1.1.3 持久化存储与存档机制

为了保障玩家进度的安全性，项目引入了企业级的数据持久化方案。

(1) ORM 框架与数据库集成

采用 `Hibernate` 作为对象关系映射框架，结合轻量级的 `SQLite` 数据库，设计了 `GameSaveEntity` 数据模型，实现了游戏核心数据向关系型数据库的平滑落地。

(2) 游戏状态序列化与读写

开发了基于 DTO（数据传输对象）的序列化机制（GameStateDTO），允许玩家自定义存档名称，随时保存当前的角色位置、属性、背包内物品以及各房间散落物资的状态，并支持动态加载恢复。

1.2 任务目的

本课程实践不仅是为了完成一个可运行的游戏项目，更是为了在一个高仿真、全链路的软件工程场景中，全面锻炼团队的工程化思维与技术落地能力。主要目的包括：

1.2.1 掌握面向对象设计原则与设计模式应用

(1) 运用命令模式优化系统结构

针对原有代码中臃肿的 if-else 指令判断，通过引入命令模式（Command Pattern），将 go、take、drop、eat、back 等操作抽象为独立的命令类。

(2) 提升代码可扩展性

遵循“开闭原则（OCP）”与“单一职责原则（SRP）”，实现高内聚、低耦合的系统架构，使团队在后期扩展新指令或新机制时，无需修改核心引擎代码。

1.2.2 体验全栈开发与前后端协同生态

(1) 熟悉前后端数据通信规范

通过自主设计 HTTP 接口与 JSON 数据契约，深入理解前后端分离架构下的状态同步问题，掌握异步请求（Fetch API）的生命周期与异常处理机制。

(2) 掌握现代前端工程化思想

跨越传统 Java 后端的舒适区，学习并实践 Vue.js 的数据双向绑定、响应式状态管理以及 CSS 动画渲染等现代 Web 前端核心技术。

1.2.3 深入理解软件工程实践与团队协作

(1) 规范化版本控制与敏捷迭代

以小组为单位，熟练使用 Git 与 GitHub 进行代码托管与版本控制。通过设立 Milestone（里程碑）、拆分 Issue、提交规范的 Commit message 以及发起 Pull Request (PR) 进行代码合并，完整体验企业级敏捷协同开发的工作流。

(2) 软件测试与质量保证

通过编写基于 JUnit 的单元测试（如 PlayerTest、RoomTest、GameHistoryTest），覆盖核心业务边界，培养测试驱动开发（TDD）的意识，保障重构与迭代过程中的系统稳定性。

二、小组任务计划

本次实践任务的核心目标是将经典的单机控制台文本游戏“World of Zuul”彻底重构，升级为一款基于 B/S（Browser/Server）架构、支持数据持久化、具备 2D 图形化界面与动态战斗系统的现代 Web RPG 游戏。为了确保项目的高效推进与技术落地，小组在开发初期对整体任务进行了深度的拆解与分析。

2.1 任务分析

本次项目的开发任务并非简单的代码平移，而是涉及底层架构变更、设计模式重构、全栈开发与 UI/UX 体验升级的综合性软件工程实践。小组将整体任务拆解为以下五个核心分析维度：

2.1.1 系统架构演进与前后端分离通信设计

传统的 Zuul 游戏为强耦合的单机应用，本次实践的首要任务是完成从单体架构向前后端分离架构的演进。

（1）轻量级 API 服务构建

由于不依赖庞大的 Spring 框架，后端需要利用 Java 原生的 HttpServer 从零搭建 Web 服务容器（GameServer）。任务重点在于设计并实现 RESTful 风格的 API 接口（如 /api/state 状态同步、/api/command 指令下发、/api/save 存档等），并处理跨域、异步非阻塞的 HTTP 请求。

（2）状态同步与 JSON 数据通信

前端可视化重度依赖后端的数据驱动。任务难点在于定义严谨的 JSON 数据契约（DTO），确保后端的复杂对象（如 Room、Player、Item）能够被高效序列化，并在前端 Vue.js 中被准确解析与渲染。

2.1.2 核心引擎的面向对象重构与业务拓展

原版 Zuu1 的指令解析往往伴随着臃肿的 if-else 结构，且玩法仅限于移动。任务重点在于应用面向对象的设计原则对核心引擎进行重构。

(1) 基于命令模式的解耦

引入命令模式 (Command Pattern)，建立 CommandExecution 执行核心，将 go、take、drop、look、eat 等行为抽象为独立的命令类。任务难点在于保证系统符合“开闭原则 (OCP)”，使得未来新增指令（如 buy、attack）时无需修改解析器核心层。

(2) RPG 机制与多维状态封装

在基础导航之上，扩展角色模型（增加 HP、MP、负重属性）与物品模型（区分可食用道具、装备武器等）。任务需要重点处理对象间的交互逻辑，例如拾取物品时的负重校验、食用魔法饼干后的状态恢复等业务边界问题。

2.1.3 2D 可视化前端与图形渲染机制

突破命令行黑底白字的限制，构建极具代入感的 2D RPG 视图是前端的核心任务。

(1) 现代响应式 UI 框架应用

任务需要熟练运用 Vue 3 的 Composition API（响应式状态 reactive、ref）进行全局状态管理，并结合 Bootstrap 5 快速构建控制面板与模态框（存档/读档弹窗）。

(2) 游戏主循环与雪碧图动画解析

为了实现流畅的人物走动与战斗动作，任务难点在于在浏览器端建立基于 requestAnimationFrame 的游戏主渲染循环 (Game Loop)。同时，需要开发 SpriteAnalyzer 模块，动态计算和切割人物雪碧图 (Sprite Sheet)，结合键盘事件监听实现 WASD 的平滑移动与边界碰撞检测。

2.1.4 数据持久化与 ORM 框架集成

让游戏具备存档、读档功能是本次实践提升软件工程实用性的关键，要求实现内存状态向物理存储的落地。

(1) 轻量级关系型数据库引入

选用 SQLite 作为嵌入式数据库，降低部署成本的同时满足数据持久化需求。

(2) 对象关系映射（ORM）配置

引入 Hibernate 框架进行数据层操作。任务的重难点在于设计合理的数据实体表结构（GameSaveEntity），并通过自定义序列化方案，将复杂的当前游戏上下文（玩家状态、背包列表、全地图物品分布、当前所在房间等）转换为 GameStateDTO 并以 JSON 格式无损存储至数据库，确保读档时能 100% 还原现场。

2.1.5 动态交互系统与沉浸式体验优化

在基础的探索功能之上，构建一个具有自我闭环的 RPG 生态是本阶段的核心进阶任务。

(1) 多维度 NPC 交互与 AI 状态机

任务要求设计具有不同职责的 NPC 系统。包括具有自主巡逻与攻击 AI 的敌对 NPC（Saber），以及具有交互反馈的友善 NPC。重点与难点在于利用前端渲染帧与后端指令系统的配合，实现如“实验室博士（Doctor）”的多阶段任务状态机（questPhase：寻找、收集、完成），推动主线剧情发展。

(2) 经济模型与资源调度

引入货币（Coins）与经济消耗机制。任务要求在后端增加全新的购买指令（CommandBuy），并在前端的“校园酒吧（Pub）”场景中设计“酒吧老板（Barmaid）”的交易面板，实现玩家通过任务获取金币，再消耗金币

购买属性恢复道具（如恢复 HP 的麦脉暖酿、恢复 MP 的月花蜜醴）的完整经济闭环，极大增强了游戏的可玩性与资源管理深度。

2.2 项目架构与功能设计

本项目彻底摒弃了原版“World of Zuul”单体、同步、基于控制台的简单架构，采用现代化的 B/S (Browser/Server) 架构与前后端分离的开发模式，构建了一个具有完整数据持久化层、业务逻辑层和可视化展现层的 Web RPG 游戏系统。

2.2.1 总体架构设计

系统整体划分为三层架构，各层之间通过标准化的协议与数据格式进行解耦与通信：

（1）前端展现层

基于 Vue 3 (Composition API) 构建，负责 2D 游戏地图渲染、雪碧图(Sprite)动画解析、用户键盘/点击事件捕获、以及游戏主渲染循环(Game Loop) 的运行。

（2）后端业务层

不依赖臃肿的 Web 框架，使用 Java 原生 HttpServer 搭建轻量级 Web 容器。负责接收前端的 RESTful API 请求（如 /api/command、/api/state），解析并执行核心游戏逻辑，维护游戏世界（Map、Player、Room、NPC）的全局状态。

（3）数据持久层

集成 Hibernate ORM 框架与轻量级 SQLite 数据库，负责将内存中复杂的对象网络（如背包列表、房间状态）序列化为 DTO（数据传输对象），并持久化到物理存储中，提供多档位的存取支持。

2.2.2 后端核心架构与设计模式应用

后端引擎在保留 Zuu1 核心地图节点（Room）流转思想的基础上，进行了深度的面向对象重构：

（1）基于命令模式（Command Pattern）的解耦

系统设计了 Command 抽象父类与 CommandExecution 调度核心，将每一种玩家操作（如 CommandGo, CommandTake, CommandEat）封装为独立的策略类。由 Parser 负责解析用户输入的字符串并映射到对应的命令对象。这种设计完美符合开闭原则（OCP），极大提升了指令系统的可扩展性。

（2）轻量级 RESTful API 网关

通过 GameServer 类实现请求路由，制定了严格的跨域策略（CORS）与异步非阻塞的通信机制。后端将游戏状态封装为统一的 JSON 结构下发，确保前端视图的响应式更新。

2.2.3 前端视图与交互引擎架构

前端不仅仅是数据的展示，更是 2D 游戏引擎的承载体，具备以下架构特点：

（1）游戏主循环与物理帧更新

利用浏览器的 requestAnimationFrame API 构建了每秒 60 帧的 gameLoop。该循环负责处理 WASD 平滑位移、NPC AI 寻路、魔法子弹弹道计算以及状态锁定（如战败/弹窗时的冻结）。

（2）基于 DOM 的 2D 图形渲染栈

通过分层思想（z-index），将地图背景（.room-container）、地面掉落物（.room-item）、角色模型（.player-character）与 UI 状态条（.player-ui-layer）进行解耦渲染。

（3）雪碧图异步解析系统（SpriteAnalyzer）

自主封装了图片像素级分析器，动态计算并切割人物的 Sprite Sheet，

通过修改 CSS background-position 配合 @keyframes 动画，实现了流畅的行走与攻击动作反馈。

2.2.4 核心功能模块设计

围绕 RPG 游戏的核心体验，本项目在功能层面设计了以下五大模块：

(1) 探索与物理碰撞模块

①**二维网格移动系统**：角色坐标采用百分比（0-100%）表示，支持全向平滑移动。

②**高精度碰撞检测**：针对每个房间（如剧院、酒吧、实验室）定义了专属的静态碰撞盒（Bounding Boxes，见 ROOM_COLLISIONS），通过两点距离公式与边界补偿算法（pushOutOfCollision），实现阻挡与滑墙效果。

③**智能边界预警机制**：计算玩家坐标与门界面的距离（WARN_DIST），当玩家靠近出口处时，系统结合当前所在房间自动推算出目标场景名称，并在角色头顶以中文气泡形式进行预告提示。

(2) 角色与物品状态管理模块

①**多维角色属性**：设计了包含 HP（生命值）、MP（魔法值）、基础攻击力、动态攻击力与背包负重（MaxWeight）的完整人物模型。

②**物品交互引擎**：环境物品具备重量、名称与交互效果。拾取时不仅会触发专属音效，还进行严格的背包容量与负重校验；支持消耗品系统（如“魔法饼干”恢复 HP）和装备系统（如“道具剑”增加攻击力）。

(3) 动态战斗与任务系统模块

①**独立 NPC AI 状态机**：设计了守护者 NPC “Saber”。通过定时器（saberMoveTimer）实时计算与玩家的欧几里得距离，实现警戒触发、自动追击与近战伤害判定。

②**双规战斗机制**：提供近战物理攻击（J 键）与消耗 MP 的远程魔法

子弹（Q 键）。引入了战败重置惩罚与击杀胜利结算机制（恢复满血满蓝状态）。

③**主线任务校验器**：定义了 `requiredItems` 任务清单（如收集碎片、钥匙），击败 Boss 时系统将自动校验背包库存，条件不满足则无法通关，增强了探索的驱动力。

（4）存档与数据持久化模块

①**DTO 序列化引擎**：由 `GameStateDTO` 统筹打包游戏的“快照”，包括玩家状态、动态变化的背包物品、各个房间内被改变位置及数量的物品清单。

②**多档位数据库管理**：利用 `Hibernate` 将 `JSON` 序列化后的快照与时间戳、玩家姓名、自定义存档名（`GameSaveEntity`）一并写入 `SQLite` 数据库，并提供 `UI` 面板支持一键读取（`Load`）与覆盖。

（5）沉浸式 `UI` 与全局本地化模块

①**复古羊皮纸视窗**：抛弃原生 `HTML` 提示框，使用自定义无边框羊皮纸背景图承载系统消息。构建了双层包装（`.sys-message-content`）彻底解决长文本在 `Flex` 布局下的断行错乱与行距问题。

②**全局拦截与中文化转译**：由于后端核心代码部分保留了英文环境，前端系统在 `sendCommand` 方法中建立了一张绵密的“正则拦截网”。针对后端的各种反馈（如 `cannot carry any more`, `not in the room` 等边界警告）以及房间地名，100% 截获并替换为带有特殊古风字体（`ActionFont/BodyFont`）与色彩样式的纯正中文气泡，实现了视觉表现层面的高度沉浸感。

2.3 团队组成与分工

本项目代码量大、业务逻辑复杂，且涉及前后端分离的全栈开发。为

确保项目高效高质量落地，本小组基于敏捷开发（Agile Development）思想，依托 GitHub 平台构建了规范的代码协作流。团队成员根据个人技术特长进行了明确的角色定位与模块分工，确保“人人有事做，事事有落实”。

2.3.1 小组协作模式与管理策略

在具体分工前，小组确立了以下协同开发机制：

（1）扁平化敏捷管理

设立项目负责人（组长）统筹全局，但技术决策由团队共同评审。采用 Milestone（里程碑）机制进行冲刺（Sprint）规划，将大需求拆解为独立的 Issue。

（2）基于 GitHub 的版本控制流（Git Flow）

团队成员禁止直接向 master/main 分支提交代码。开发新功能时需拉取独立的特性分支（如 feat-ui-upgrade、feat-npc-economy），开发完成后发起 Pull Request（PR），经代码 Review 无严重冲突后方可合并，并使用 Fixes #Issue 编号 语法自动追踪任务进度。

2.3.2 团队成员与具体分工说明

（1）组长范奕轩：项目经理兼后端架构核心开发

主要负责统筹项目进度、搭建后端底层框架以及网络通信层的构建。

① **项目管理与基建：**负责 GitHub 仓库的初始化、.gitignore 配置、Maven 依赖管理（pom.xml 的编写与自动化打包脚本测试），以及各阶段 Milestone 的任务分配。

② **原生 Web Server 搭建：**使用 Java HttpServer 搭建轻量级非阻塞式的游戏服务容器。开发并映射 /api/command、/api/state 等核心 RESTful 接口，实现跨域支持与前后端数据通信。

③ **命令模式底层重构：**负责对原版 Zuu1 核心引擎进行面向对象重构，建立 CommandExecution 接口与注册表体系，将解析器（Parser）

与具体命令（如 Go, Quit, Help）解耦，为后续扩展打下基础。

（2）组长范奕轩+队员王启赢：前端工程与 UI/UX 沉浸式体验开发

主要负责 2D 可视化游戏引擎的搭建、界面美化设计及全局本地化工作。

①Vue 3 视图层与游戏主循环构建：利用 Vue 3 的 Composition API 维护复杂的前端状态树，利用 requestAnimationFrame 构建 60FPS 的游戏渲染主循环（Game Loop），实现平滑的 WASD 移动与多维边界碰撞检测。

②沉浸式 UI 重构与双轨字体集成：摒弃传统 Web 控件，设计并实现了自适应宽度的“无边框做旧羊皮纸”游戏消息面板，解决了多行长文本的 Flex 排版错位问题。引入并加载了定制的书法体（ActionFont）与手写体（BodyFont）。

③全局中文语境封装与边缘提示：在前端编写了复杂的正则拦截器，100% 拦截并翻译后端抛出的底层英文错误与状态；开发了基于真实物理坐标计算的“靠近门气泡预警”系统，以及精美的战败/战胜结算动态弹窗。

（3）队员何宇航：游戏系统工程师与业务逻辑开发

主要负责 RPG 核心玩法的扩展、NPC 交互生态、经济与任务系统的开发。

①复杂业务指令扩展：在后端核心的命令注册表中，深度扩展了 CommandTake（拾取）、CommandDrop（丢弃）、CommandEat（食用）、CommandBuy（购买）等高阶业务指令，并严密处理了背包容量与物品负重上限的业务边界。

②多阶段任务与经济系统构建：独立开发了位于计算机实验室的“博士（Doctor）”NPC 及三阶段主线任务状态机（寻找→收集→领赏）；开发了位于校园酒吧的“酒吧老板（Barmaid）”交易系统，引入金币（Coins）属性，打通了获取金币与购买补给（如麦脉暖酿、月花蜜醴）的经济内

循环。

③**动态战斗与 AI 引擎开发**：构建了“Saber”敌对 NPC 的自动寻路、警戒距离计算及攻击频率控制（异步定时器）。配合开发了前端的魔法子弹（Q 键发射）与近战判定（J 键挥砍）的战斗交互系统。

（4）**队员王启赢+队员何宇航：数据库工程师与软件质量保证(QA)**
主要负责游戏进度的持久化存储落地以及系统的自动化单元测试。

①**ORM 框架与 SQLite 数据库集成**：配置 Hibernate 引擎与嵌入式 SQLite 数据库，建立 GameSaveEntity 数据实体。通过设计高效的数据传输对象（DTO，包含 GameStateDTO, ItemDTO, RoomDTO），实现了复杂游戏上下文（玩家、物品位置、地图状态）的 JSON 序列化与反序列化。

②**存档管理与状态恢复接口**：编写后端的 SaveHandler 与 LoadHandler，实现了多档位的数据覆盖与读取逻辑，并对旧版本存档结构兼容（如默认坐标归位兜底）进行了深度修复。

③**软件测试驱动**：负责编写基于 JUnit 的单元测试类（如 PlayerTest, RoomTest, GameHistoryTest），对核心类的边界条件（如超重拾取、非法移动路线）进行代码级断言验证，保障重构过程中的软件质量。

2.4 软件工具与项目约束

为了保障全栈开发的高效推进，同时贯彻现代软件工程中敏捷开发与 DevOps 的核心理念，本小组在项目启动之初便明确了开发与协作所需的一系列软件工具，并制定了严格的项目管理策略与强制性代码约束。

2.4.1 开发与工程化工具使用方案

针对前后端分离的架构设计与持续集成的需求，团队精选了以下软件

工程工具：

（1）后端开发与数据持久化工具

① **Java Development Kit (JDK) 8**：作为后端核心逻辑开发的基石，利用其原生 `HttpServer` 提供轻量级 Web 服务。

② **Apache Maven**：作为项目构建与依赖管理工具。通过配置 `pom.xml`，统一管理 JUnit、Hibernate、SQLite-JDBC 以及 Gson 等第三方依赖，并配置 `maven-jar-plugin` 插件实现项目的自动化编译与可执行 JAR 包的打包。

③ **Hibernate ORM & SQLite**：利用 SQLite 作为轻量级嵌入式关系型数据库，结合 Hibernate 框架进行对象关系映射，实现游戏状态的 JSON 序列化落地与持久化存储。

（2）前端视图与交互开发工具

① **Vue 3 (Composition API) & Bootstrap 5**：通过 CDN 引入 Vue 3 构建前端响应式数据引擎，管理复杂的玩家状态与游戏数据流；引入 Bootstrap 5 提供标准化的模态框（存档/读档）与基础 UI 组件。

② **HTML5/CSS3 & JavaScript (ES6+)**：使用原生 JS 构建 `SpriteAnalyzer` 逐像素解析精灵图（雪碧图）；利用 CSS3 的 `@keyframes` 动画与 `filter: drop-shadow` 等高级特性，打造高度沉浸式的做旧羊皮纸 UI 与 2D 图形渲染。

（3）DevOps、版本控制与辅助开发工具

① **Git & GitHub**：作为代码版本控制与团队远程协作的核心平台。利用 `.gitignore` 过滤编译产物与环境配置文件。

② **GitHub Issues & Milestones**：作为敏捷项目管理的看板工具。通过建立 Milestone（如 `milestoneV2.0`）规划版本周期，利用 Issue 拆解开发任务（如“新增经济系统”、“UI 重构”）并指派给具体成员。

③ **GitHub Actions (CI/CD)**: 遵循任务要求, 配置自动化流水线。在代码推送到远程仓库时, 自动触发代码格式规范检查与 JUnit 测试用例验证, 并在通过后自动执行 Maven 脚本完成项目的打包发布。

④ **IDE 与 AI 辅助编程工具**: 后端主要使用 IntelliJ IDEA 进行 Java 开发与调试; 前端主要使用 VSCode / Cursor 结合 AI 辅助模型 (基于项目要求的 AI 辅助开发许可), 提升重复性代码 (如正则匹配拦截) 的编写与排错效率。

2.4.2 项目管理策略与强制约束

为了避免多人协作引发的代码冲突和质量失控, 团队在整个生命周期内严格遵循以下项目约束:

(1) 分支管理与代码合并约束

① **Git Flow 分支模型**: 严禁任何成员直接向 master (或 main) 主分支推送 (Push) 代码。所有新功能、Bug 修复必须基于主分支拉取独立的特性分支 (Feature Branch, 如 feat-ui-upgrade)。

② **Pull Request (PR) 审查机制**: 特性分支开发完毕后, 必须在 GitHub 提交 PR 请求。经组内其他成员进行 Code Review (代码审查), 且解决所有冲突 (Conflicts) 并保持 CI 测试全绿后, 方可合并入主分支。合并时需利用 Fixes #Issue 编号 语法自动关闭关联任务。

(2) 代码编写与提交规范约束

① **Commit Message 规范化**: 严格要求每次 Git 提交必须附带具有明确语义的说明。采用 <type>: <subject> 的格式, 例如 feat: (新功能)、fix: (修复)、style: (样式调整)、chore: (构建过程或辅助工具变动, 如剔除误提交的 CLAUDE.md)。

② **代码可读性与注释约束**: 所有 Java 类与复杂业务方法必须配备符合 Javadoc 标准的块注释; 前端正则拦截器、游戏主循环 (Game Loop)

等核心算法区域必须保留清晰的业务逻辑说明，确保代码在团队内部的无障碍交接。

(3) 软件质量与集成测试约束

①测试驱动与边界约束：所有核心业务逻辑（如玩家超重判定、房间连通性、历史记录栈的回退逻辑）必须配备对应的单元测试用例（如 `PlayerTest`、`RoomTest` 等）。

②自动化测试门禁：结合 GitHub Actions 流水线，若某次 PR 或 Push 导致 JUnit 单元测试失败（如重构引发回归错误），则该次代码提交将被系统拒绝或标记为“失败”，强制要求开发者在本地修复验证后重新提交。

三、小组软件过程

在本次“World of Zuul”重构项目中，开发小组没有采用传统的“单机开发、U 盘整合”的落后模式，而是全面引入了现代软件工程中的敏捷协同（Agile Collaboration）与 DevOps 思想。小组以分布式版本控制系统为基础，构建了一套涵盖任务分配、分支开发、代码审查、持续集成到打包发布的全生命周期软件开发流程。

3.1 代码仓库

代码仓库是整个团队协作运转的“中枢神经”。为了保障多人在全栈开发过程中的代码安全与版本一致性，小组在项目初期对代码仓库进行了严谨的规划与初始化配置。

3.1.1 平台选择与仓库初始化

本项目采用 Git 作为底层分布式版本控制工具，并选择 GitHub（结合 GitHub Classroom）作为云端代码托管与协作平台。

（1）仓库建立

在 GitHub 上建立了小组专属的私有/内部代码仓库，确保所有组员拥有统一的 Remote 目标（Origin）。

（2）工程描述文件（README.md）

在仓库根目录规范化编写了 README.md 文档。该文档不仅包含了项目的背景介绍、核心功能（RPG 玩法、2D 可视化）、技术栈说明，还详细记录了本地运行与编译指南，作为新成员或代码审查者的第一手参考资料。

（3）忽略清单配置（.gitignore）

为保持远程仓库的纯净度，小组精心配置了 .gitignore 文件。严格拦截了 target/ 编译目录、.idea/ 等 IDE 缓存文件、SQLite 的运行期本地日志，以及通过协作审查发现并剔除的误提交文件（如 CLAUDE.md），

从源头上杜绝了代码库的冗余与合并时的环境冲突。

3.1.2 仓库目录规范与架构映射

为了适应全栈开发模式，小组在仓库内部制定了严格的目录组织规范，使代码结构与物理架构完美映射：

(1) **src/main/java/**：后端核心 Java 源码区。内部严格按照包名（如 `cn.edu.whut.sept.zuul`）组织，进一步划分为 `db`（数据库持久层模块）、命令模式核心库及 Web API 处理器层，保障了高内聚低耦合。

(2) **webapp/**：前端静态资源与视图区。集中存放了 `index.html`（Vue 3 视图主入口）、`images/`（雪碧图、羊皮纸 UI 及场景贴图）、`fonts/`（定制古风字体）以及 `sounds/`（交互音效），实现了前后端静态资源的物理隔离。

(3) **src/test/java/**：单元测试用例专区。集中存放基于 JUnit 编写的各种测试脚本（如 `PlayerTest`、`RoomTest`）。

(4) **pom.xml & 构建脚本**：存放在根目录的 Maven 配置文件，统筹管理第三方依赖库并定义了构建打包生命周期；结合 `rebuild.bat` 等辅助脚本，为团队提供了一键构建环境。

3.1.3 权限管控与安全策略

为防止误操作导致核心代码被覆盖或污染，小组在 GitHub 仓库中启用了严格的权限管控机制：

(1) **角色分配**：由项目组长担任仓库的 Administrator/Maintainer，负责仓库全局设置、Milestone（里程碑）管理与核心分支保护规则的制定；其他组员作为 Developer 参与开发。

(2) **主分支保护**对 master 主干分支实施了锁定策略。任何成员（包括组长）均被禁止通过命令行直接向主分支执行 `git push` 操作。所有对主干分支的修改必须通过 Pull Request (PR) 形式提交，为后续的代码 Review

和自动化门禁测试提供了先决条件。

3.2 软件版本计划

面对从单机纯文本程序跨越到全栈 Web 2D RPG 游戏的巨大工程量，开发小组摒弃了传统的瀑布开发模型，全面采用敏捷迭代（Iterative Development）策略。项目生命周期被科学地拆解为 5 个核心里程碑（Milestones）。每个里程碑都包含一组高度内聚的 Issue 任务，确保每个阶段结束时，团队都能交付一个可运行、可测试的软件版本。

3.2.1 五大里程碑拆解与实施目标

(1) Milestone 1: 基础架构重构与前后端互通

①**核心目标**: 完成单机系统向 B/S 架构的转型，搭建底层通信法则。

②**关键任务**: 在后端引入命令模式（Command Pattern），解耦核心解析器；搭建原生 Java HttpServer，制定 /api/command 与 /api/state 等 RESTful 接口契约。前端基于 Vue 3 完成初始化构建，实现基本的指令下发与状态接收。

③**交付标准**: 前后端成功连通，玩家可以通过 Web 页面发送基础指令（如 go、help）并获得状态回显。

(2) Milestone 2: 2D 可视化地图与物理引擎核心

①**核心目标**: 突破文本限制，建立 2D 游戏坐标系与图形渲染栈。

②**关键任务**: 前端开发 SpriteAnalyzer 雪碧图解析器，实现角色的多向行走动画；引入基于 requestAnimationFrame 的游戏主循环（Game Loop）；设计二维网格坐标系，并在各房间（如实验室、酒吧）建立高精度碰撞检测盒（Bounding Boxes）。

③**交付标准**: 玩家可以通过 WASD 键在地图中平滑移动，触发跨房间传送，且不会穿透墙壁等障碍物。

(3) Milestone 3: 物品交互系统与数据持久化核心目标:

①**核心目标:** 丰富场景互动, 并保障玩家游戏进度的安全落地。

②**关键任务:** 扩展后端 take、drop 指令, 完善背包容量与负重上限逻辑; 集成 Hibernate ORM 框架与 SQLite 数据库; 设计 GameStateDTO, 实现包含玩家位置、背包明细及全图物品分布的复杂状态 JSON 序列化与读写。

③**交付标准:** 实现环境物品的拾取与丢弃, 并提供多档位的游戏存档与读档面板。

(4) Milestone 4: RPG 生态构建——NPC、经济与任务系统

①**核心目标:** 为游戏注入灵魂, 打造具有自我驱动力的 RPG 业务闭环。

②**关键任务:** 开发“酒吧老板”NPC, 新增 buy 指令与金币系统, 实现属性补给品的交易内循环; 开发“博士”NPC, 构建寻找、收集、交付的三阶段主线任务状态机; 开发敌对 NPC “Saber” 的自动寻路 AI 与双轨战斗系统 (近战与魔法子弹)。

③**交付标准:** 形成“接取任务→探索收集→换取金币→购买补给→挑战 Boss”的完整游戏生态体验。

(5) Milestone 5: 沉浸式 UI 重构、测试与集成发布

①**核心目标:** 打磨最终产品质量, 实现表现层的高度本地化与自动化部署。

②**关键任务:** 重构无边框做旧羊皮纸 UI, 引入 ActionFont 与 BodyFont 双轨自定义古风字体; 建立前端正则拦截器, 实现系统警告与提示的 100% 纯正中文化气泡渲染; 编写 JUnit 核心单元测试; 配置 GitHub Actions 实现自动化测试拦截与 Maven 自动化打包。

③**交付标准:** 所有测试用例全绿通过, UI 沉浸感拉满, 最终产出可

独立分发运行的打包工程文件。

3.2.2 基于 Milestone 的版本控制机制

在具体的协作过程中，小组通过 GitHub 平台将上述 5 个里程碑作为版本迭代的“锚点”。每个 Milestone 均设定了明确的截止日期(Due Date)，并在其名下绑定了多个细粒度的开发任务 (Issues)。

团队成员严格遵循特性分支 (Feature Branch) 工作流，针对分配到的 Issue 建立独立分支开发。代码编写完成后发起 Pull Request (PR) 并关联 Issue 编号，在经过代码 Review 并成功合并入主干后，对应的 Issue 自动关闭，Milestone 的进度条随之推进。这种基于里程碑的跟踪机制，极大地保障了项目进度的透明度与交付质量。

3.3 开发分支模型与提交流程

在多人协同的全栈开发环境下，前后端代码的频繁变动极易引发代码冲突和系统崩溃。为此，小组在项目初期便确立了严格的分支管理策略与代码提交流程，以保障开发的高效并行与主线代码的绝对稳定。

3.3.1 基于成员实名的专属分支模型

考虑到本小组共有 3 名核心成员，且各自负责前后端不同的核心业务模块，团队没有采用过于繁琐的传统 GitFlow 模型，而是量身定制了一套“基于成员实名的专属分支模型”。该模型既保证了敏捷性，又实现了物理层面的工作区隔离：

(1) 主干分支(master)

作为受保护的基准分支，代表了游戏当前最稳定的、可运行的 Release 状态。严禁任何成员直接在本地对 master 分支进行 commit 与 push 操作。

(2) 个人开发分支(FanYixuan / WangQiyang / HeYuhang)

三位组员分别拥有以自己姓名拼音命名的专属远程分支。

①**范奕轩 (FanYixuan)**：主要在该分支上推进前端 Vue 3 引擎的迭代、UI 羊皮纸视窗重构以及全局中文正则拦截等视觉与物理层工作。

②**王启赢 (WangQiyang) & 何宇航 (HeYuhang)**：主要在各自的分支上推进后端架构演进、数据库 DTO 序列化封装以及复杂 NPC（酒吧老板、博士）任务与经济系统等核心业务逻辑的开发。

(3) 隔离与协同机制

每位成员每天在自己的专属分支上进行代码编写与测试，互不干扰。当一个功能模块（如“完成酒吧老板购买指令”）在个人分支上测试通过后，再通过规定的合并流程同步至 master。

3.3.2 语义化与原子化的代码提交流程

为了让项目的演进历史清晰可溯，小组严格贯彻了语义化提交规范（Conventional Commits）与原子化提交（Atomic Commits）原则。根据小组的项目提交记录，整个提交流程体现出极高的工程素养：

(1) 规范化的 Commit 标识前缀

团队约定，每一次 git commit 的 Summary（标题）必须以特定的英文前缀开头，以便快速界定本次代码变更的性质：

①**feat(新功能)**：用于提交全新的业务逻辑或机制。例如记录中的 feat：新增战斗胜利结算弹窗与全局 UI 沉浸式汉化重构，清晰表明了 RPG 闭环系统的新增。

②**style(样式与视图优化)**：用于处理前端 UI/UX 的视觉变动。例如记录中的 style：优化游戏指南 UI，实现透明书本边缘的真实投影与融合，表明本次提交未修改后端底层逻辑，仅针对表现层进行了滤镜（drop-shadow）重构。

③**chore(杂项与工程配置)**：用于提交与业务功能无关的构建配置或

工程化清理。例如记录中的 chore: 移除误提交的 CLAUDE.md 并更新 gitignore, 记录了团队对代码库纯净度的一次维护操作。

④fix(漏洞修复): 用于记录在测试阶段被发现并解决的 Bug 补丁。

(2) 结构化的 Description (描述详情)

在标题之外, 小组成员还会在 Commit 的详细描述区 (Description) 分点列出本次提交的具体改动项。这种习惯使得代码 Review 时, 评审者能够瞬间理清代码的上下文逻辑, 降低了沟通成本。

3.3.3 标准代码合并闭环 (Merge Workflow)

结合小组的分支模型, 团队制定了每日的代码集成标准作业程序 (SOP):

①同步主线 (Fetch & Pull): 成员在本地开发前, 必须先执行 `git pull origin master`, 将主干上的最新代码拉取并合并到本地的个人分支 (如 FanYixuan) 中。

②本地冲突解决 (Resolve Local Conflicts): 如果在拉取主线代码时发现与自己编写的代码产生冲突, 需在本地 IDE 中进行冲突解决, 确保项目能够顺利编译运行。

③推送到远程专属分支 (Push to Origin): 提交本地 Commit 后, 推送到远程仓库的同名专属分支。

④发起与完成合并 (Pull Request & Merge): 在 GitHub 网页端发起将个人分支合并到 master 的 Pull Request。由项目负责人或其他交叉评审的组员确认无误后, 执行 Merge 闭环。

通过这套“专属分支+语义化提交+标准合并”的组合拳, 小组成功在整个开发周期内保持了代码库的高度整洁与稳定运转。

3.4 代码规范与检查流程

在前后端分离的多人协同开发模式下, 统一的代码风格与严格的代码

质量检查是保障系统可维护性和团队协作效率的生命线。为此，小组建立了一套“规范先行、人工审查、机器兜底”的立体化代码规范与检查流程。

3.4.1 统一的代码编写规范

针对 Java 后端与 Vue/JS 前端不同的技术生态，小组分别制定了严格的代码编写规范，确保不同成员编写的代码风格保持一致：

（1）命名规范

①**后端（Java）**：严格遵循阿里巴巴 Java 开发手册规范。类名采用大驼峰命名法（UpperCamelCase，如 GameServer、TransporterRoom）；方法名与变量名采用小驼峰命名法（lowerCamelCase，如 processCommand）；常量统一使用全部大写并以下划线分隔（如 AUTO_SAVE_NAME）。

②**前端（Vue/JS/CSS）**：JavaScript 变量与函数采用小驼峰命名法；CSS 类名严格采用连字符命名法（kebab-case，如 player-character、guide-overlay），以保持样式表的高可读性。

（2）注释与文档规范

①**JavaDoc 标准**：对于后端所有的核心业务类（如 Parser、Player）及公共方法，强制要求编写标准的 Javadoc 块注释，明确标明 @param（参数含义）、@return（返回值）及方法功能。

②**内联注释**：对于前端复杂的算法模块（如 SpriteAnalyzer 中的像素级碰撞计算、pushOutOfCollision 中的安全推离算法），必须在代码块上方保留清晰的逻辑说明注释，降低后续维护和交接的阅读成本。

（3）架构边界规范

严禁越权调用。后端代码只负责业务逻辑与数据持久化，必须通过统一的 DTO（如 GameStateDTO）序列化为 JSON 返回，严禁在后端硬编码拼接 HTML。前端仅负责视图渲染与事件分发，严禁在前端直接操作底

层文件系统。

3.4.2 人工交叉代码审查机制

代码合入主干(master)前,必须经过严格的人工审查(Code Review),以发现自动化工具难以察觉的业务逻辑漏洞和架构缺陷:

(1) PR (Pull Request) 强制评审: 组员在完成专属分支的开发后,通过 GitHub 发起 PR。PR 必须指定至少一名其他核心成员作为 Reviewer。

(2) 审查维度:

①逻辑完整性: 如新增 CommandBuy 指令时,是否处理了“金币不足”、“物品不存在”等边界异常。

②性能与安全: 检查数据库操作(DatabaseManager)中是否正确使用了 try-finally 块,确保 Session 和 Transaction 能够被安全关闭,防止内存泄漏。

③UI/UX 表现: 前端提交的 PR 需提供截图或动态演示,审查者需确认 UI 在不同尺寸屏幕下是否会发生排版错乱。

(3) 意见反馈与修正: Reviewer 在 GitHub 对应的代码行留下评审意见(Comments),提交者需根据意见进行代码修改并重新 Push,直至获得 Approve 标识方可合并。

3.4.3 CI/CD 自动化代码门禁与测试检查

为贯彻 DevOps 理念并落实课程任务要求,小组充分利用了 GitHub Actions 构建了自动化的代码集成流水线(CI Pipeline),实施了严格的代码门禁策略:

(1) 自动化格式检查

流水线配置了代码格式检查插件。当成员推送代码时,系统会自动扫描代码格式是否符合预设的语法规则(如检查多余的空格、未使用的变量等)。对于严重违规的代码,流水线将抛出错误。

（2）自动化单元测试拦截：

每次 PR 提交或 Push 操作，GitHub Actions 将自动执行 Maven 的 `mvn clean test` 生命周期。系统会拉起所有的 JUnit 测试用例（包括 `PlayerTest` 负重测试、`RoomTest` 边界测试等）。

（3）门禁阻断机制

如果在自动化流程中，出现格式检查不通过或任何一个测试用例失败（显示为红色的），GitHub 系统将自动锁定合并按钮（Merge pull request 被禁用）。开发者必须在本地修复 Bug 并补充提交，直到流水线恢复全绿状态（显示为绿色的），才允许将代码正式并入主分支。这种机制彻底杜绝了“带着 Bug 上线”的风险。

3.5 项目测试计划与实施流程

在敏捷开发与重构的过程中，为防止新功能的引入破坏原有逻辑（即避免回归缺陷，Regression Bugs），本小组严格贯彻了测试驱动开发（TDD）与自动化测试的理念。团队制定了详尽的软件测试计划，并依托 JUnit 测试框架，构建了从底层单元测试、模块集成测试到系统端到端测试（E2E）的立体化质量保障体系。

3.5.1 测试策略与工具选型

（1）测试分层策略

遵循经典的“测试金字塔”模型，小组将测试重点放在底层核心逻辑的单元测试上（占比最高），其次是数据库与 API 层的集成测试，最后是模拟玩家操作链路的系统端到端测试。

（2）测试工具链：

①**核心框架：**选用 JUnit 4/5 作为 Java 后端自动化测试的核心框架，利用 `@Test`、`@Before`、`@After` 等注解管理测试生命周期。

②**构建与执行**：通过 Maven Surefire Plugin 将测试用例无缝集成到构建生命周期中。成员只需在本地运行 `mvn clean test` 即可一键拉起全量测试。

③**CI/CD 拦截**：结合 GitHub Actions，规定任何特性分支在提交 PR 时，必须 100% 通过远端的自动化测试用例，否则合并将被强制阻断。

3.5.2 核心模块单元测试实施 (Unit Testing)

单元测试的重点在于验证系统中最小可测试单元的正确性。小组针对实体模型和命令解析器编写了覆盖率极高的测试用例类：

(1) 实体模型与边界值测试

针对游戏核心对象 (Player、Room、Item)，团队实施了严格的边界值分析 (Boundary Value Analysis)。

①**PlayerTest & PlayerFullTest**：重点测试了角色状态管理与背包机制。涵盖了 HP/MP 增减的边界拦截（如血量不能为负）、物品拾取时的负重超限抛出异常测试、以及背包物品的准确查找与移除。

②**RoomTest & RoomFullTest**：验证了房间出口映射 (Exits Mapping) 的正确性，以及散落物品在房间内部的动态添加与移除逻辑是否符合预期。

(2) 命令模式与解析器测试

针对基于设计模式重构的指令系统，团队为每一个具体指令都编写了专属的测试类，确保“开闭原则”的平稳落地。

①**ParserTest & CommandWordsTest**：验证了字符串输入被正确拆分为合法指令对象的能力，以及非法指令的容错与阻断处理。

②**动作指令测试集**：如 `CommandTakeTest`、`CommandDropTest`、`CommandEatTest`，重点验证了物品在“玩家背包”与“房间地面”之间的流转逻辑，以及“食用魔法饼干”带来的 HP 恢复效果。

③**业务指令测试集**：特别针对新增的经济系统，编写了 CommandBuyTest，严密测试了“玩家金币扣除”、“商店物品发放”以及“金币不足/物品不存在时的交易回滚”逻辑。

3.5.3 集成测试与系统端到端测试 (Integration & E2E Testing)

为了确保各模块组合后能够协同工作，小组在单元测试之上，进一步实施了数据库集成测试与全链路流程模拟测试：

(1) 数据持久层集成测试

数据的无损存取是 RPG 游戏的命脉。团队编写了以下测试类保障数据流转：

①**GameStateDTOTest & GameSerializeTest**：验证了复杂的运行期内存对象（如包含海量物品的地图、玩家的属性与坐标）能够被精准地转换为 DTO 数据传输对象，并成功序列化为合法的 JSON 字符串。

②**HibernateUtilTest & DatabaseManagerTest**：测试了 ORM 框架的初始化加载，以及 SQLite 数据库的 CRUD 流程，确保存档（Save）能够正确落盘，且读档（Load）时能够完美重建玩家的历史上下文。

③**GameServerTest**：对轻量级 HttpServer 的 API 网关进行了测试，验证了跨域请求配置（CORS）与状态回显接口的高可用性。

(2) 游戏业务流端到端测试 (E2E Test)

为了模拟真实玩家的游玩体验，团队专门设计了高度仿真的业务流程测试，验证各子系统的耦合协同能力：

①**GameNewGameTest & GameWorldTest**：验证了整个游戏地图拓扑结构的初始化、出口双向连通性以及系统默认配置的加载。

②**GameQuestTest**：专门针对实验室“博士（Doctor）”的三阶段主线任务状态机进行系统级验证。模拟了玩家“接取任务→跨房间收集指定物品→返回提交任务→验证系统回收物品并正确发放金币奖励”的完

整链路。

③ **GameTeleportTest & GameHistoryTest**：验证了神秘传送室（Transporter Room）的随机传送机制，以及利用 CommandBack 指令进行历史路径多级回退的栈操作逻辑。

3.5.4 测试评估与质量保障机制

通过上述极为全面的测试类（共计 20 余个专属 Test Class），小组实现了对游戏核心引擎的高代码覆盖率（Code Coverage）。

在实际开发流程中，测试体系发挥了巨大的兜底作用。例如，在后期重构经济系统与负重系统时，最初的代码改动曾导致物品掉落逻辑出现空指针异常，而 CommandDropTest 瞬间捕捉到了该回归缺陷并在本地构建时发出报错警报，使得开发者能够将 Bug 消灭在提交代码之前。严密的测试计划不仅大幅减少了后期系统联调的痛苦，更铸就了版本发布时的强大信心。

3.6 项目集成与发布规范

在现代软件工程中，代码编写完成并不意味着项目的结束，如何将前后端代码、静态资源与数据库配置安全、稳定地集成，并交付给终端用户，是项目落地的关键。为打通研发到交付的“最后一公里”，小组建立了一套基于持续集成（CI）与自动化构建的项目集成与发布规范。

3.6.1 持续集成流水线

为防止“在我的电脑上能跑，但在你电脑上报错”的“集成地狱”现象，团队摒弃了手动整合代码的原始方式，全面依托 GitHub Actions 部署了自动化的持续集成流水线。

（1）自动化构建触发机制

系统配置了监听器，当任何成员向 GitHub 远程仓库的 master 分支

发起 Pull Request (PR) 或进行 Push 时，云端自动化流水线将被立即触发。

(2) 标准作业流

①**环境初始化**：在云端容器中自动拉取最新源码，并配置与生产环境一致的 Java 运行环境 (JDK 8+)。

②**依赖拉取与编译**：执行 Maven 引擎，根据 pom.xml 自动下载包含 Gson、Hibernate、SQLite-JDBC 在内的所有第三方核心依赖，并对项目源码进行全量编译。

③**单元测试拦截 (Test Gate)**：自动拉起所有的 JUnit 测试用例执行测试驱动。若任何一个测试未通过 (如抛出断言异常)，集成流程将立刻中断，拒绝代码合入主分支。

3.6.2 前后端全栈构建与打包策略

本项目的特殊性在于它是一个不依赖 Tomcat 或 Spring 庞大框架的轻量级 Web 全栈应用。为实现“开箱即用”的用户体验，小组在构建策略上进行了精细的设计：

(1) 静态资源无缝集成

前端 Vue 3 视图文件、样式表、定制字体 (.ttf) 与图片 (.png/.jpg) 统一收拢于项目根目录的 webapp/文件夹下。后端的 GameServer 实现了专门的文件路由映射，确保打包后，Java 原生 HttpServer 能够直接向浏览器下发这些静态 Web 资源，实现了前后端在运行期的完美集成。

(2) Maven 自动化打包引擎

通过配置 Maven 构建插件 (如 maven-shade-plugin 或 maven-assembly-plugin)，在执行 mvn clean package 时，将所有编译后的 .class 文件及庞大的第三方依赖库打包成一个可执行的胖包 (Fat JAR)。

(3) 批处理脚本辅助 (Build Script)

为方便团队内部在本地进行快速联调测试，项目中编写并维护了 `rebuild.bat` 脚本，开发者双击即可一键清理缓存、重新编译打包并拉起服务端进程，极大提升了本地集成的效率。

3.6.3 零配置部署与版本交付规范

在版本交付与用户部署层面，小组追求最高程度的“极简部署”，降低用户的上手门槛。

（1）语义化版本控制（Semantic Versioning）

项目的交付严格遵循 主版本号. 次版本号. 修订号（如 `v1.0.0`, `v2.1.0`）的命名规范。每次完成 Milestone 冲刺后，会在 GitHub 发布区（Releases）打上相应的版本 Tag。

（2）轻量级零配置环境：

①**数据库免安装**：得益于物理架构设计，系统引入的是 SQLite 嵌入式数据库。用户无需额外安装 MySQL 或配置繁杂的数据库账户，系统启动时 Hibernate 会自动检查并生成相应的数据库文件及表结构。

②**一键运行方案**：终端用户在获取发布包后，只需系统内配有 Java 环境，执行简单的 `java -jar` 命令即可启动游戏引擎。随后，用户打开本地任意现代浏览器，访问 `http://localhost:端口号` 即可看到沉浸式的游戏画面。

（3）发布文档规范（Release Notes）

每次对外发布 Release 版本时，必须随包附带详尽的更新日志（Changelog）与 README.md 手册，说明本次版本新增的 NPC、战斗特性以及已知问题的修复情况。

四、小组成员任务报告

每次对外发布 Release 版本时，必须随包附带详尽的更新日志（Changelog）与 README.md 手册，说明本次版本新增的 NPC、战斗特性以及已知问题的修复情况。

4.1 组长：范奕轩 任务分工及实施过程描述

4.1.1 承担的主要任务分工

在本项目中，本人担任项目组长及前端与地图引擎负责人。核心目标是将基于命令行的纯文本冒险游戏彻底重构为现代化的 2D RPG Web 图形游戏。本人全面接管了表现层（View Layer）的架构与物理交互引擎的落地，同时统筹了项目的初期基建。具体承担的任务包括：

（1）项目基建与架构初始化

引入并整合底层命令模式体系、领域模型（Player/Room/Item）与基于原生 JDK 的 HTTP 服务器骨架，完成单体应用向 B/S 架构的底层过渡。

（2）2D 渲染与物理引擎开发

基于 Vue 3 构建单页应用，开发高精度 CSS 雪碧图动画引擎（Sprite Engine），实现基于百分比的二维物理坐标系与边界碰撞系统。

（3）场景交互与特殊机制支持

设计“靠近门景”的视觉透视与方向探测体系，并独立开发神秘法阵的触发与随机传送系统。

（4）沉浸式 UI 重构与视觉包装

摒弃标准 Web 控件，利用 CSS 滤镜技术打造透明 3D 投影效果，构建胜利结算等动态弹窗体系。

（5）全局语境本地化（汉化）

全面推翻原版的英文回显，实现 100% 纯正中文 UI 与系统消息，并引入

自订古风字体对操作标识与正文进行视觉隔离。

4.1.2 实施过程描述

(1) 前端工程初始化与雪碧图渲染引擎构建

为实现流畅的 RPG 走动体验，本人放弃了简单的 CSS 过渡，转而在浏览器中自主建立了一套前端渲染引擎。

①SpriteAnalyzer 算法研发：针对角色和 NPC 的 4×4 帧网格雪碧图 (Sprite Sheet)，本人编写了原生的 Canvas 图像解析算法。通过在内存中读取图像的 `naturalWidth/naturalHeight`，逐帧从底部向上扫描 Alpha 透明通道 (判定 `alpha > 128` 的有效像素)，动态计算出精确的底部边界 (`avgFootRatio`)。这使得游戏能够通过精确的 `translateY` 偏移将角色双脚牢牢锚定在地面上，消除了常见的“滑步”与“悬空”感。

②多态动画控制：利用 CSS 的 `background-position` 实现方向映射 (如左转对应 33.33%，上行对应 100%)，并专门编写了 `@keyframes attack-swing` 关键帧动画，将挥剑动作精准拆分为蓄力 (0-25%)、挥砍 (闪白, 25-50%) 与收刀渐隐 (50-100%) 三个视觉阶段。

(2) 物理坐标系与动态场景探测机制

①绝对坐标与边界碰撞：设计了一套 0-100 的百分比物理坐标系统。为 6 个不同长宽比的房间加载独立的全屏全景图 (`object-fit: cover`)，并在代码层设定了严格的绝对边界 (`EDGE_MIN=4`, `EDGE_MAX=96`)，确保角色无法走出视口。

②门景探测与气泡预警：在前端主循环中注入了实时坐标探测器。当角色移动到房间边缘的特定“门区域” (距离边界 4% 以内，容差 $\pm 7.5\%$) 时，系统会动态比对当前房间的数据拓扑，于角色头顶弹出精美的漂浮气泡 (应用 `@keyframes bubble-float` 动画)，提前预告即将前

往的区域（如“前往：剧院大厅”）。

③神秘法阵空间交互：独立开发了 `TransporterRoom` 相关的空间判定。当玩家进入坐标（35-65, 35-65）的法阵中心时，前端会拦截原生操作并向后端发送隐藏指令 `trigger_teleport`，随后调用 `getRandomRoom()` 触发全局随机传送机制。

（3）沉浸式 UI/UX 深度打磨与排版优化

①全局本地化与双轨字体集成：全面梳理了前后台交互的提示文案，将所有系统界面、操作按钮、提示标签 100% 中文化。为了区分“系统动作”与“剧情描述”，通过 `@font-face` 引入了手写体 `ActionFont`（针对高亮动作如“[拾取]”）与书法体 `BodyFont`（针对普通描述），极大增强了 RPG 游戏的古典代入感。

②滤镜融合技术应用：在重构“新手指南”等浮层 UI 时，发现传统的 `border-radius` 与矩形 `box-shadow` 会导致透明背景的书本 PNG 图像产生严重的“矩形发光”割裂感。本人通过剥离常规边框属性，大胆引入了高级 CSS 滤镜 `filter: drop-shadow()`。该滤镜成功识别了书本不规则的破损边缘，投射出逼真的 3D 悬浮阴影，实现了界面与暗黑地图的完美融合。

③胜利结算状态机：设计了与任务系统高度耦合的结算弹窗（`showVictoryDialog`）。当系统判定 Boss 被击败且全部 4 件任务物品集齐时，自动挂起游戏主循环，弹出带有满血满蓝奖励的结算浮层。

（4）数据流联调与疑难数据修复

①反序列化坐标漂移修复：在联调存读档功能时发现，旧版本存档在反序列化后会导致物品坐标全部归零 (0,0)。本人在 `Game.deserializeGameState()` 的重建生命周期中介入，建立了一套包含 17 种物品的关键词坐标映射表。一旦检测到异常坐标，系统会自动依据

物品的 description 子串进行精准匹配并恢复其在地图中的预设物理位置。

②**背包交互坐标同步**：修复了通过背包 UI 面板直接拖拽丢弃物品时，坐标未同步至后端的 Bug。重构了 CommandDrop 的参数解析逻辑，确保前端的鼠标落点百分比坐标能够正确写入数据库。

4.1.3 实施成果展示

经过上述工程实施，项目不仅顺利完成了底层的全栈架构转型，更交付了一个视觉表现力极强、交互丝滑的 Web 前端产品：

(1) 成功突破了原生浏览器的 DOM 渲染瓶颈，实现了稳定 60FPS 的角色移动、场景切换与动画解析，地图与 NPC 判定毫无卡顿。

(2) 彻底清除了程序的“Web 网页感”，通过自适应羊皮纸面板、透明阴影投影以及定制字体的融合，呈现出一款极具质感的复古 2D 冒险游戏。

(3) 建立了一套高可用的底层坐标与动画接口，为队友后续接入“Boss 战斗引擎”与“酒水商店 NPC”提供了坚实的前端图形基座。

4.2 成员二：王启赢 任务分工及实施过程描述

4.2.1 承担的主要任务分工

在本项目中，本人主要担任后端交互逻辑与系统特性工程师。核心目标是打通前端视图与后端逻辑的交互壁垒，并为 2D 游戏世界注入真实的物理碰撞规则与沉浸式听觉反馈。具体承担的任务包括：

(1) 物理碰撞与安全引擎开发

构建基于 AABB（轴对齐包围盒）与圆形的混合碰撞检测系统，实现角色在地图障碍物边缘的平滑滑动与防卡死推离。

(2) 沉浸式动态音效系统（Audio Engine）

基于 HTML5 Audio API 集成 BGM 与交互音效，并创新性地研发了“基

于物品材质特征匹配”的分类掉落音效系统。

(3) 快捷交互指令流与数据同步

重构物品交互逻辑，绑定快捷键（F 拾取、G 丢弃、E 使用），并确保图形界面的所有动作均能精确穿透至后端并触发 SQLite 数据库的自动存档（Auto-Save）。

(4) 主线任务驱动与随机收集系统

完善“探索与收集”的主线业务循环，构建动态的候选任务物品池及进度追踪机制。

4.2.2 实施过程描述

(1) 高精度物理碰撞与滑动响应机制（Collision System）

原版的网页地图是完全开放的，玩家会穿透背景图中的墙壁和桌子。为解决此问题，本人在前端独立开发了一套物理引擎模块：

①**多态碰撞盒定义**：为游戏内的 5 个具有实体障碍的房间（如剧院、酒吧、实验室）分别建立 ROOM_COLLISIONS 静态矩形碰撞数组，采用百分比坐标系精准描绘场景中的吧台、实验桌与观众席。

②**分轴检测与滑动响应算法**：在计算玩家（圆形碰撞体， $PLAYER_RADIUS = 4.5$ ）与障碍物（矩形）的碰撞时，采用 AABB vs Circle 算法（即寻找矩形上距离圆心最近的点，判断其距离平方是否小于半径平方）。为了让玩家碰到墙壁后不会被完全“死锁”，本人创新性地使用了“分轴响应逻辑”：在 GameLoop 中，先让 X 轴位移并检测碰撞（若撞墙则只回退 X 轴），再让 Y 轴位移并检测碰撞。这使得玩家按住斜方向键时，能够沿着墙壁顺滑地“滑动”，极大提升了操作手感。

③**安全推离兜底（Push Out）**：针对传送或读档后玩家可能被错误生成在墙体内部的极少数异常情况，编写了 pushOutOfCollision 函数。一旦检测到重叠，系统会计算最短法线距离，瞬间将玩家沿着合法方向

推离出障碍区域，确保游戏主流程永不卡死。

（2）按材质分发的动态音效系统构建

①**基础音频架构：**在项目中挂载原生的 `<audio>` 标签并封装 JS 控制器，实现了背景音乐（BGM）的弱音量（0.3）无缝循环播放以及全局操作的拾取音效（pickup.mp3）。

②**基于正则匹配的材质掉落反馈：**为了增强物品交互的拟真感，本人摒弃了千篇一律的丢弃声音。通过编写 `getDropSoundId` 判定树，根据玩家丢弃的物品名称（`itemName`）进行特征提取。例如，检测到“烧杯”、“试剂瓶”、“玻璃”等关键词时，路由并播放清脆的玻璃碎裂声（drop-glass.mp3）；检测到“铁锹”、“显微镜”时，播放沉闷的金属撞击声（drop-metal.mp3），为系统赋予了极强的细节生命力。

（3）键盘快捷交互与全链路数据同步

①**快捷按键监听与动作节流：**在前端封装了原生的 `keydown` 事件，将玩家常用的鼠标操作映射为高频快捷键（如 F 键探测周边 10% 范围内的物品并自动拾取，G 键将物品精确丢弃在玩家当前的百分比坐标点上）。

②**全链路状态一致性保障：**为确保“所见即所得”，本模块强制规定所有前端发起的物理动作，都必须经过 REST API（`/api/command`）下发至 Java 后端。后端执行对应的 `CommandTake` / `CommandDrop` 策略，修改内存数据后立即触发 `autoSave()` 落盘至 SQLite 数据库。待收到 HTTP 200 成功响应后，前端才拉取最新状态进行视图刷新，从架构上彻底杜绝了前后端状态撕裂的风险。

（4）随机任务生成与状态流转

在 `Game.java` 中实现并配合前端重构了任务分发系统。每次开局，系统会从多达 17 种场景物品池（如鹅卵石堆、飞镖盘等）中随机洗牌

(Collections.shuffle)，抽取出 4 件绝对不重复的物品作为本局的收集目标，结合前端的任务追踪面板，极大丰富了游戏的 Rogue-like 随机探索体验。

4.2.3 实施成果展示

(1) 成功在 Web 前端建立了一套对标独立 2D 游戏标准的物理引擎碰撞体验，零穿模、零卡死。

(2) 声音系统与画面操作达到了极高的同步率，多材质掉落音效成为了本项目在细节展现上的重大加分项。

(3) 彻底打通了复杂物品交互在“前端视图→后端业务→数据库物理存储”的无损数据链路，玩家的所有高频微操（甚至掉落在地面的精确坐标）均能被安全存档。

4.2.4 AI 辅助开发工具使用说明

在开发高强度的物理逻辑与繁琐的分类系统时，本人借助大语言模型 (AI) 显著提升了研发效能：

(1) 算法思路生成与调优：在最初构思“如何判断一个圆和一个矩形是否碰撞”时，利用 AI 辅助查阅了游戏开发领域经典的 AABB-Circle 算法，并请 AI 协助将该算法的数学公式翻译为适配本项目百分比坐标系的 JavaScript 代码，节约了大量查阅公式的时间。

(2) 字典匹配优化：在编写材质音效的关键词判定表时，将庞大的物品池提供给 AI，让其迅速帮我生成了一套结构清晰、无遗漏的 JS 正则匹配字典 (Array.includes() 封装)，避免了繁琐的手工枚举。

4.3 成员三：何宇航 任务分工及实施过程描述

4.3.1 承担的主要任务分工

在本项目中，本人主要担任数据库工程师与业务系统（战斗/任务）开发负责人。核心目标是保障游戏复杂运行期数据的安全落地，并为地图探索框架注入灵魂，打造出具有自我驱动力的 RPG 生态系统。具体承担的任务包括：

（1）ORM 架构与数据持久化

引入 Hibernate 框架与嵌入式 SQLite 数据库，设计序列化方案，彻底解决游戏存读档的数据落地难题。

（2）RPG 双轨战斗引擎与 AI 状态机

设计敌对 NPC（Saber）的自我寻路 AI，并开发玩家的近战、远程魔法子弹及状态回复战斗模组。

（3）经济模型与主线任务链闭环

独立开发“酒吧老板”商店交易系统（引入金币概念）以及“实验室博士”的三阶段主线收集任务系统。

（4）快捷交互面板开发

在 UI 侧构建支持数字键快捷选中的 3 槽位道具快捷栏，并实现其与核心背包状态的实时双向联动。

4.3.2 实施过程描述

（1）基于 Hibernate + SQLite 的数据持久化与防嵌套序列化

由于单机版的内存状态极其复杂（涉及相互引用的房间拓扑图、背包与地图物品分布），直接存库难度极大。本人实施了以下架构方案：

①自定义 SQLite 方言 (Dialect) 适配：为了让 Hibernate 5.6 完美支持无需安装的本地 SQLite 3.44 数据库，本人编写了

SQLiteDialect 类，精准映射了 `BOOLEAN→integer`、`CLOB→text` 等数据类型，并自定义了主键自增策略（`select last_insert_rowid()`）。

②安全的数据管理网关：封装了单例模式的 `HibernateUtil` 控制 `Session` 生命周期，并编写 `DatabaseManager` 实现了 7 个标准 CRUD 接口。为优化性能，在存档列表查询 `listSavesMeta()` 中使用了 HQL 投影查询，避免拉取庞大的 JSON 字段。

③基于 DTO 的无损快照技术：为了防止序列化时的无限循环引用（如房间 A 指向房间 B，B 又指向 A），设计了解耦的 `GameStateDTO` 数据传输对象。通过五步重建法（先建房间→连通出口→放置物品→重建玩家→压入历史栈），实现了复杂游戏状态向 JSON 文本的高效双向转换。

（2）高并发战斗引擎与动态 Boss AI 搭建

战斗系统的引入彻底改变了游戏的玩法性质，需要在原有回合制文本指令基础上构建具有实时属性的系统：

①独立 AI 定时器引擎：为 Boss（Saber）编写了异步 AI 循环（`setInterval 500ms` 评估一次）。根据欧几里得距离动态切换状态——距离 $> 8\%$ 触发寻路追击算法（计算坐标差向量）；距离 $\leq 8\%$ 则触发伤害高达 10 点的近战攻击，并配合前端播放 `saber_attack` 雪碧图动画。

②双轨资源池与魔法弹道系统：建立了独立的 HP（生命值）/ MP（法力值）资源模型。除了支持基础的近战挥砍（J 键，依据手中是否装备“道具剑”判定 10 或 20 伤害），还实现了复杂的远程魔法子弹（Q 键，消耗 10MP）。魔法子弹按每帧独立计算飞行弹道，并在撞击墙壁碰撞盒或命中 Saber 时触发动态销毁逻辑。

（3）三阶段任务状态机与经济内循环闭环

①NPC 交互状态机 (State Machine)：在实验室场景创建“博士”NPC，并通过 questPhase 变量维护了极其严密的三阶段状态机 (find_doctor 接取任务→collect_items 收集检验→completed 交付奖励)。玩家只有集齐了开局随机分布的 4 件任务物品才能完成交付，从而触发 100 金币的进账。

②商品购买机制与拦截：结合新引入的货币系统，在酒吧场景开发了“酒吧老板”NPC，并配套在后端扩展了 CommandBuy 指令。实现了“金币校验→物品分发→负重上限校验→扣除金币”的严谨事务逻辑，允许玩家购买“麦脉暖酿”与“月花蜜醴”进行战中补给，完美闭合了“收集-变现-消费-战斗”的经典 RPG 经济链。

(4) 基于 Vue 响应式的动态快捷栏系统

为降低战斗中的操作成本，本人在屏幕右下角构建了 3 槽位的快捷面板：

①通过监听 state.inventory 的变化，实现 refreshQuickSlots() 方法的自动调用，将背包前 3 件物品同步映射至快捷栏。

②封装数字键 1/2/3 的动态高亮切换事件以及 R 键的快速使用事件。结合 canEat 属性判断，实现一键回复状态，极大地提升了紧张战斗下的操作流畅度。

4.3.3 实施成果展示

(1) 成功将系统的存取逻辑推向了企业级标准，用户实现了“开箱即用、自动建库”，并且具备了可靠的 Auto-Save (自动存档) 能力。

(2) 将原本单一的“走路模拟器”升级为具有高度可玩性、充满挑战性的硬核战斗 RPG 游戏，AI 的压迫感与魔法技能的释放均达到了极佳的表现效果。

(3) 经济、商店与主线任务链的无缝衔接，使得玩家的游戏目标极其

明确，大幅提高了项目的完整度与趣味力。

4.3.4 AI 辅助开发工具使用说明

在构筑复杂的架构层代码与业务状态逻辑时，本人在一定程度上借助了大语言模型提升开发效率：

(1) 框架环境配置攻坚：在使用 Hibernate 桥接非常规的 SQLite 时遇到众多方言 (Dialect) 报错。利用 AI 模型辅助定位了底层的映射异常，并迅速生成了诸如 IdentityColumnSupport 等高度复杂的底层重写代码，节约了数天的排错时间。

(2) DTO 序列化防死循环策略：在最初存入数据库时，Gson 遭遇了房间互相引用的内存溢出问题。利用 AI 进行架构探讨，最终敲定了剥离原生对象、构建包含 ItemDTO 和 RoomDTO 的结构树方案，优雅地化解了深度嵌套引发的栈溢出崩溃。

五、实施过程问题记录与分析

在本次“World of Zuul”重构项目的实施过程中，开发小组经历了从传统单体控制台应用向现代 B/S 架构全栈图形化 Web 游戏跨越的技术挑战。由于引入了自定义物理引擎、复杂状态机以及底层数据持久化机制，团队在协作与开发中遇到了多维度的技术瓶颈。以下是对实施过程中典型问题的记录与深度分析：

5.1 前端图形渲染与物理引擎异常分析

5.1.1 透明 PNG 图像边缘渲染发光问题

在重构沉浸式“新手指南”UI 时，前端系统在渲染不规则透明背景图像时出现了视觉割裂。

（1）问题具体表现与复现场景

①**矩形发光现象**：当为边缘破损的透明古书 PNG 图片添加 box-shadow 阴影时，图片周围出现了一圈违和的发光边框，暴露了底层的 HTML 矩形容器边界。

②**边缘硬性裁剪现象**：由于父容器设置了 overflow: hidden，导致古书原本不规则的破损边缘被生硬地裁剪成了圆角矩形，破坏了游戏古典 UI 的沉浸感。

（2）原因排查与技术解决方案

①**CSS 盒模型渲染机制排查**：经分析，在 Web 浏览器的 CSS 盒模型（Box Model）中，所有的阴影和边框属性默认作用于 DOM 元素的物理矩形边界上，无法直接穿透并识别 PNG 图像内部的有效像素边缘。

②**引入 drop-shadow 滤镜剥离重构**：团队对该 UI 类进行了底层属性剥离，移除了原有的 border-radius 与 box-shadow。转而引入了高级 CSS3 滤镜 filter: drop-shadow()。该滤镜能够自动识别 PNG 图像的

Alpha 透明通道，顺着古书内部不规则的像素边缘投射出真实的 3D 立体悬浮阴影，完美实现了 UI 与暗色地图环境的视觉融合。

5.1.2 物理碰撞引擎与场景边缘探测脱节

前端独立渲染的物理坐标系与后端的业务逻辑模型在联调时出现了边界死锁与数据不匹配问题。

(1) 碰撞死锁与分轴响应重构

①**对角线卡墙死锁现象：**在引入 ROOM_COLLISIONS 静态碰撞盒后，当玩家控制角色按住对角线方向（如同时按 W 和 D）撞向吧台或墙壁时，角色会瞬间停止移动，仿佛被“粘”在墙上，导致操作手感极其僵硬。

②**分轴滑动响应算法的落地：**初版的 AABB 碰撞算法采用“一碰即停”的简单逻辑。为此，团队重构了物理碰撞响应：将移动拆解为 X 轴和 Y 轴两次独立运算。先沿 X 轴移动，若发生碰撞则仅取消 X 轴位移；随后再沿 Y 轴移动并检测。这一算法使得角色在撞墙后，能够顺着墙壁边缘自然“滑动”，彻底解决了卡墙死锁 Bug。

(2) 预警气泡数据脱节与物理寻址解耦

①**后端字典强耦合导致的气泡失效：**在开发“靠近门气泡预警”功能时，最初设计直接读取后端的出口 JSON 数据。但由于后端保留了原版英文底层逻辑，返回的字段（如 east）与前端需要的纯正中文地名（如“剧院大厅”）无法对齐，导致气泡经常显示空白。

②**基于百分比坐标与状态机的独立推导：**团队放弃了强依赖后端出口字段的方案，转而采用“物理坐标推导算法”。前端在 gameLoop 主循环中直接计算玩家当前坐标与四面门边界（容差距离 WARN_DIST = 10%）的相对距离。一旦触碰警戒线，结合当前所在的房间标识，直接在前端使用状态机推导出目标房间的中文字符串，实现了表现层与底层数据的完美解耦。

5.2 复杂对象网络与数据持久化危机

5.2.1 Hibernate 与 Gson 序列化引发的栈溢出

单机版运行期极度复杂的内存状态在落盘至 SQLite 数据库时遭遇了严重的序列化瓶颈。

(1) 循环引用导致的 StackOverflow 崩溃

①房间出口相互引用现象：原版游戏模型设计中，“大学正门”的东出口对象指向“剧院大厅”，而“剧院大厅”的西出口对象又指回“大学正门”。

②Gson 深度遍历栈溢出原因：在执行自动存档（Auto-Save）时，Gson 引擎试图将完整的 Game 对象序列化为 JSON。由于存在上述相互依赖的环状网络，Gson 在解析时陷入了无限递归的深搜循环，直接抛出 StackOverflowError 并导致后端容器崩溃。

(2) 引入 DTO 模式的安全重建策略

①剥离原生模型构建 GameStateDTO：后端成员引入了数据传输对象（DTO）设计模式。构建了独立于业务模型的 GameStateDTO（内部嵌套轻量级的 RoomDTO 和 ItemDTO）。在序列化时，主动打断循环链条，仅记录房间 ID 和相邻出口的字符串映射关系。

②反序列化五步安全重建法则：在读档时，设计了严密的五步重建生命周期：先实例化所有独立房间→恢复出口的内存连通性→放置并初始化物品→重建玩家及背包状态→恢复历史回退栈。该方案优雅地化解了深度嵌套引发的落地难题。

5.2.2 历史存档反序列化后的坐标漂移 Bug

测试团队在进行回归测试时，发现了旧版本存档读取后的地图物品渲染异常。

(1) 物品坐标全局归零异常

①旧存档读取后物品堆叠于(0, 0)：加载早期版本的存档后，地图中的物品全部重叠渲染在房间左上角的 (0, 0) 坐标处，由于偏离了正常活动区域，玩家无法触发拾取交互。

②无参构造器生命周期漏判原因：早期的存档 JSON 未包含物品的具体百分比坐标，导致 Gson 在反序列化时触发了 Item 类的无参构造器，成员变量 x 和 y 被默认赋予初始值 0.0。

(2) 坐标兜底修复与向下兼容设计

①异常坐标拦截机制：在反序列化数据回填的生命周期中，加入了一层严格的拦截校验拦截器。一旦检测到物品坐标为 $ix == 0.0 \ \&\& \ iy == 0.0$ ，系统将判定该物品数据发生异常或来自旧版本存档。

②基于 Description 的关键词预设映射恢复：触发拦截后，系统将提取该物品的 description 子串，并通过一张硬编码的关键词映射表为其重新分配预设的物理位置（例如，包含“显微镜”的物品被强制映射归位至 (25, 55) 坐标点），从而保障了历史版本数据对最新引擎的向上兼容。

5.3 敏捷协同与版本控制冲突事故

5.3.1 多分支错误回退与环境文件污染

在项目冲刺阶段，由于小组成员频繁进行前后端代码合入，引发了严重的版本控制事故。

(1) Git Revert 误删与配置文件提交事故

①误用 revert 导致多媒体文件诡异丢失：在合入音效系统(Issue #21)至 master 主干时引发了代码冲突。某成员错误执行了 Git revert 撤销操作，在未充分比对差异的情况下强行提交，导致仓库中原有的部分 UI 样式及音频文件（如 drop-glass.mp3）被连带删除，引发运行时资源 404 错

误。

②IDE 配置文件污染远程主干：开发期间，部分由本地编辑器与 AI 辅助工具产生的运行时文件（如 CLAUDE.md 及 SQLite 产生的 .db-shm 缓存日志）被未经筛选地 `git add .` 全部提交至主分支，造成了代码库环境的严重污染。

（2）事故紧急修复与长效防范策略

①分支比对与手工补录修复：事故发生后，项目组长紧急锁定主干代码，召集团队比对 Git Commit 树状记录。由相关责任人切出 hotfix 修复分支，手工将被误删的文件重新补入并提交合并，使项目快速恢复了稳定编译状态。

②.gitignore 升级与主干强制锁配：为杜绝此类事故再次发生，小组升级了根目录下的 .gitignore 文件，将所有数据库日志与辅助配置文件列入黑名单。同时在 GitHub 仓库设置中开启了 Branch Protection Rules（分支保护规则），禁止任何人对 master 执行直接推送与强推（Force Push），所有变更必须经由 Pull Request 和全量单元测试通过后方可合入。

六、任务总结

本次《软件工程实践》课程的任务二，是一次极具挑战性且意义深远的综合性开发实践。开发小组没有局限于对样例工程“World of Zuul”进行简单的控制台指令扩充，而是以企业级软件工程的标准，将其彻底重构为一款具备 B/S 架构、2D 图形界面、数据库持久化与动态战斗系统的现代 Web RPG 游戏。

6.1 项目总体完成情况

对照实训任务书的需求指标，本小组不仅 95%完成了所有基础与进阶要求，更在系统的可玩性与工程复杂度上实现了质的飞跃：

（1）基础需求全覆盖

完成了基于 Player 类的玩家模型重构，实现了随身物品及负重管理（take/drop/items 指令）；完善了基于栈的多级回退逻辑（back）；实装了具备空间随机传送能力的魔法阵房间（TransporterRoom），以及通过魔法饼干（eat）提升负重的机制。

（2）架构与表现层重构

成功突破了单机终端的限制，搭建了 Java HttpServer + Vue 3 的前后端分离架构。前端不仅实现了 6 个房间的全屏背景映射，更独立开发了高精度的 CSS 雪碧图动画引擎与 AABB 物理碰撞系统，呈现出极具沉浸感的 2D 互动画面。

（3）业务生态的闭环构建

在探索玩法之外，从零构建了基于金币（Coins）的酒水商店交易系统、基于“博士”NPC 的三阶段主线任务系统，以及基于“Saber”AI 的动态双轨战斗（近战+魔法子弹）与结算系统，赋予了游戏完整的 RPG 生态灵魂。

6.2 技术能力与工程实践收获

在本次项目的高强度冲刺中，小组成员在软件架构设计与工程化落地能力上获得了巨大的成长：

（1）深化了对面向对象与设计模式的理解

在扩展系统指令（如新增 buy、eat）时，团队深刻体会到了“命令模式（Command Pattern）”带来的优势。遵循开闭原则（OCP），我们无需修改核心解析器，只需新增具体的命令实现类并注册，极大降低了代码的耦合度。

（2）掌握了全栈开发与前后端通信原理

通过弃用庞大的框架，使用底层 Java 原生 API 构建 Web 容器，团队透彻理解了 HTTP 协议、跨域资源共享（CORS）以及 RESTful API 的设计规范。前端利用 JSON 承载状态数据驱动 Vue 视图更新，彻底打通了数据双向绑定的链路。

（3）践行了测试驱动开发（TDD）理念

本项目编写了多达 29 个测试类、涵盖 249 个测试方法的自动化测试集（覆盖率超 90%）。在重构复杂的数据库反序列化（DTO 转换）与边界物理碰撞算法时，这些全绿通过的单元测试成为了我们大刀阔斧修改代码的最大底气，让我们真正理解了“测试是保障软件质量的第一防线”。

6.3 团队协同与项目管理经验

现代软件开发绝不是单打独斗，本次实践让我们切身体会到了 DevOps 与敏捷管理的威力：

（1）规范化的 Git Flow 分支协作

团队采用了基于成员实名的专属分支模型。通过严格的 Pull Request (PR) 审查和“禁止向 master 直接推送”的门禁策略，保障了主干代码的

绝对稳定。

（2）拥抱自动化流水线（CI/CD）

利用 GitHub Actions 构建了自动化构建与测试拦截网。任何引发回归缺陷的代码均会在云端被阻截，同时结合 Maven 自动化打包脚本，实现了从代码提交到可执行 JAR 包输出的无缝衔接。

（3）在冲突与事故中成长

在经历过一次误操作导致音效文件丢失的 Git revert 事故后，团队迅速完善了 .gitignore 规则，并学会了如何通过 Commit 树追溯和手工合并来挽救代码库。这次挫折极大地提升了团队应对工程危机的风险管理能力。

6.4 不足与未来展望

尽管项目取得了丰硕的成果，但受限于开发周期，系统仍存在一定的优化空间：

（1）多端状态同步问题

当前 HTTP HttpServer 采用的是基于请求-响应（Request-Response）的短链接模式。未来如果向“网络多人游戏模式（任务要求 6）”演进，需引入 WebSocket 全双工通信，以支持多个玩家在同一房间内的实时坐标同步与状态广播。

（2）AI 逻辑的深度扩展

目前 Saber 敌对 NPC 的 AI 逻辑基于简单的欧几里得距离寻路与定时器触发。未来可引入 A*（A-Star）寻路算法配合 Navigation Mesh（导航网格），使其能够绕过复杂的室内障碍物进行更智能的追击。

【结语】

回首这些天的工作历程，宛如在浩瀚的代码宇宙中经历了一场奇幻的冒险。我们亲手将一个只有黑底白字、几百行代码的简陋命令行程序，一行行敲打、重塑，最终赋予了它斑斓的 2D 视界、严密的物理法则与跳动的 RPG 灵魂。屏幕上跃动的剑光与魔法，是我们在深夜里反复核对 API 接口的执着；那 249 个在流水线上全绿通过的测试用例，是我们面对底层架构重构时，恪守软件工程底线的勋章。

“World of Zuul” 不仅是我们笔下的游戏世界，更是我们三人团队探索软件工程之路的真实写照。在这场名为“开发”的试炼中，我们面对的“Boss”是盘根错节的嵌套序列化危机，是令人抓狂的物理碰撞死锁，也是频频告警的 Git 分支冲突。但幸运的是，我们拥有并肩作战的彼此——前端的绚丽渲染、后端的严密架构与数据库的坚实底座在这里完美交汇，化作了我们击碎一切技术壁垒的利刃。

这次软件工程实践，不仅是一次技术深度的蜕变，更是一场团队精神的洗礼。当看到游戏打包运行成功，当清脆的胜利提示音在屏幕前响起，所有的疲惫都在那一刻化为了无与伦比的成就感。

告别祖尔的秘境，我们的征途才刚刚开始。带着这份对代码逻辑的敬畏、对工程规范的坚守，以及在无数个 Bug 前淬炼出的默契，我们将从这间小小的计算机实验室出发，迈向真正广阔的企业级软件研发星海。代码不息，探索不止！

【参考文献】

- [1] 李晓蕙.网页游戏中界面设计的研究和应用[D].上海交通大学,2008.
- [2] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- [3] Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.
- [4] 黄敏珍.CMMI、敏捷开发和 DevOps 在项目管理实践中的应用[J].项目管理技术,2020,18(9):91-95.
- [5] Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd Edition). Addison-Wesley.
- [6] Vue3 详细教程:
https://blog.csdn.net/m0_55734030/article/details/127719835?fromshare=blogdetail&sharetype=blogdetail&sharerId=127719835&sharerefer=PC&sharesource=2403_88186781&sharefrom=from_link
- [7] Oracle. (2023). Java HTTP Server API Documentation. Retrieved from
<https://docs.oracle.com/en/java/javase/17/docs/api/jdk.httpserver/com/sun/net/httpserver/HttpServer.html>
- [8] 戴建国,赵庆展,郭理,等.持续集成在项目开发中的应用研究[J].计算机工程与设计,2009,30(10):2573-2576.DOI:10.16208/j.issn1000-7024.2009.10.045.
- [9] You, E. (2023). Vue.js Documentation - Introduction. Retrieved from
<https://vuejs.org/guide/introduction.html>
- [10] Beck, K., & Gamma, E. (2023). JUnit 4 Documentation. Retrieved from <https://junit.org/junit4/>
- [11] Jenkins CI/CD 流水线实战:
https://blog.csdn.net/weixin_45130293/article/details/144965697?fromshare=blogdetail&sharetype=blogdetail&sharerId=144965697&sharerefer=PC&sharesource=2403_88186781&sharefrom=from_link
- [12] Mozilla Developer Network. (2023). Fetch API - Web APIs. Retrieved from
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- [13] 戴启铭,毛润丰,黄璜,等.DevSecOps:DevOps 下实现持续安全的实践探索[J].软件学报,2021,32(10):3014-3035.DOI:10.13328/j.cnki.jos.006276.
- [14] Conventional Commits. (2023). Conventional Commits 1.0.0. Retrieved from
<https://www.conventionalcommits.org/>
- [15] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

- [16] Sommerville, I. (2015). Software Engineering (10th ed.). Pearson.
- [17] 邹益胜,丁国富,许明恒,等.实时碰撞检测算法综述[J].计算机应用研究,2008,(1):8-12.
- [18] Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress.
- [19] SQLite 入门教程（非常详细）零基础入门到精通，收藏这一篇就够了：
https://blog.csdn.net/2401_84618514/article/details/145874453?fromshare=blogdetail&sharetype=blogdetail&sharerId=145874453&sharerefer=PC&sharesource=2403_88186781&sharefrom=from_link
- [20] 张美香,郝轶鸣.关键帧动画技术综述[J].山西广播电视大学学报,2009,14(5):55-56.
- [21] Beck, K. (2003). Test-Driven Development: By Example. Addison-Wesley Professional.
- [22] 张扬,黄厚宽.测试驱动开发及开发实践[J].计算机技术与发展,2006,(5):74-76+79.
- [23] Nystrom, R. (2014). Game Programming Patterns. Genever Benning.
- [24] 李涛.图形用户界面 GUI 的自动测试工具的研究[D].四川大学,2005.
- [25] Bauer, C., King, G., & Gregory, G. (2015). Java Persistence with Hibernate (2nd ed.). Manning Publications.
- [26] Hibernate 使用详解：
https://techfanyi.blog.csdn.net/article/details/139897654?fromshare=blogdetail&sharetype=blogdetail&sharerId=139897654&sharerefer=PC&sharesource=2403_88186781&sharefrom=from_link

《软件工程实践（二）》成绩评定表

专业、班级	软件 2302	小组名称	longingstays
代码仓库地址	https://github.com/wutkst/kai-fa-longingstays.git		
成员一姓名	范奕轩	学号	1023005510
成员二姓名	王启赢	学号	1023004577
成员三姓名	何宇航	学号	1023004609
评价内容		满分	实得分
			得分
实践任务 完成情况	软件架构设计	25	
	软件版本计划与项目管理	25	
	软件分支管理	25	
	代码规范与代码评审	15	
	持续集成与部署	10	
实践任务评分		100	
实践报告 总评情况	学习态度与考勤	10	
	报告格式的规范性	20	
	报告的逻辑结构与语言表达	25	
	实践内容的正确性与合理性	40	
	文献引用及标注	5	
实践报告评分		100	
最终评定成绩（以优、良、中、及格、不及格评定）			

指导教师签字：

年 月 日